

COMP 300 – Assembly Language

Lab 2 – Data Declaration, Memory & System Calls

Software: MARS MIPS Simulator**Duration:** 60 Minutes

Important

Important: We will first learn how data is declared, how it appears in MARS memory, and how system calls allow our program to communicate with the user.

1. Lab Objectives

By the end of this lab, you should be able to:

1. Declare data using `.data`.
2. Declare integer variables using `.word`.
3. Declare strings using `.asciiz`.
4. Understand where declared data appears in MARS memory.
5. Use MARS system calls to:
 - print strings,
 - print integers,
 - take integer input,
 - exit the program.
6. Store user input in registers.
7. Use `add` and `sub` with values entered by the user.
8. Build a longer arithmetic expression using multiple instructions.
9. Display calculated results using meaningful prompts.

2. Part 1 – Quick Recap

From Lab 1, you already know that registers temporarily hold values.

For example:

```
li $t0, 10
li $t1, 5
add $t2, $t0, $t1
```

After execution:

Register	Value
\$t0	10
\$t1	5
\$t2	15

Today we introduce two new ideas:

- **Data declaration**
- **System calls**

We will also look at the data declared in **MARS memory**.

3. Part 2 – .data and .text

A MIPS program commonly has two important sections:

```
.data
```

and

```
.text
```

3.1 The .data Section

The `.data` section is used to declare data that your program needs.

For example:

```
.data  
  
num1: .word 10  
num2: .word 20
```

Here we have declared two integer variables.

Their labels are:

```
num1  
num2
```

and their values are:

```
10  
20
```

3.2 The .text Section

The `.text` section contains the instructions that the processor executes.

For example:

```
.text  
  
main:  
    li $t0, 5
```

So remember:

Section	Purpose
<code>.data</code>	Declare/store data
<code>.text</code>	Program instructions

4. Part 3 – Declaring Integer Variables

The directive:

```
.word
```

is used to declare a word-sized integer.

Example:

```
.data

age:    .word 20
marks:  .word 85
number: .word 100
```

We have declared three variables.

Their labels are:

```
age
marks
number
```

and their values are:

```
20
85
100
```

Important

At this stage, do **not** think of these variables as registers.
For example,

```
age: .word 20
```

does **not** mean:

```
$t0 = 20
```

Instead, the value 20 has been declared as data in memory.

5. Part 4 – Seeing Your Data in MARS Memory

This is the main purpose of today's first activity.

Create the following program:

```
.data

num1: .word 25
num2: .word 50
num3: .word 100

.text

.globl main

main:
    li $t0, 10
```

Step 1 – Assemble

Click:

Run → Assemble

or use the Assemble button in MARS.

Step 2 – Open the Memory Display

In MARS, open the memory display.

Depending on your version of MARS, the exact location/name of the memory window may differ.

Look for the memory display where addresses and their contents are shown.

Step 3 – Look for Your Declared Data

You declared:

```
num1: .word 25
num2: .word 50
num3: .word 100
```

The important idea is:

num1 → Memory → 25

num2 → Memory → 50

num3 → Memory → 100

You are **looking at the data that you declared**, without using `lw` or `sw`.

Predict Before You Run!

Before looking at memory, answer:

Variable	Expected Value	
num1	_____	25
num2	_____	50
num3	_____	100

Now inspect the memory.

Question: Did the values in memory match your predictions?

Answer: yes

6. Part 5 – Declaring Strings

We can also declare text.

Use:

```
.ascii
```

Example:

```
.data

prompt: .ascii "Enter a number: "
result: .ascii "The result is: "
```

These are strings stored in the data section.

For example:

```
prompt: .ascii "Enter a number: "
```

means that the text

Enter a number:

has been declared in memory.

7. Part 6 – Your First Data Inspection Task

Create this program:

```
.data

num1:  .word 15
num2:  .word 30
num3:  .word 45

msg1:  .ascii "First number"
msg2:  .ascii "Second number"
msg3:  .ascii "Third number"

.text

.globl main

main:
    li $t0, 5
```

Predict Before You Run!

Predict what you should find in memory.

Label	Type	Expected Value
num1	word	15 _____
num2	word	30 _____
num3	word	45 _____
msg1	string	First number _____
msg2	string	Second number _____
msg3	string	Third number _____

Now assemble the program and inspect the memory.

Task

You Try

Change the program so that:

```
num1 = 12
num2 = 24
num3 = 36
```

Then assemble the program and inspect memory again.

Did the values change?

8. Part 7 – System Calls

Now we move to one of the most important parts of today's lab:

System Calls

A system call allows your program to request a service from the simulator, such as printing something or reading input from the user.

A system call is requested using:

```
syscall
```

Before calling `syscall`, we tell MARS which operation we want by placing a number in `$v0`.

8.1 System Call 1 – Print an Integer

To print an integer:

```
li $v0, 1
li $a0, 25
syscall
```

The sequence is:

`$v0 = 1`

means:

Print integer

and:

`$a0 = 25`

means:

The integer to print is 25

Then:

```
syscall
```

performs the operation.

8.2 System Call 4 – Print a String

To print a string:

```
.data
message: .asciiz "Hello from MIPS!"

.text
```

```
main:
    li $v0, 4
    la $a0, message
    syscall
```

For printing a string:

```
li $v0, 4
```

means:

I want to print a string.

And:

```
la $a0, message
```

provides the address of the string to the system call.

Then:

```
syscall
```

performs the operation.

8.3 System Call 5 – Read an Integer

To ask the user for an integer:

```
li $v0, 5
syscall
```

The user enters a number.

MARS places the entered integer in:

```
$v0
```

For example, if the user enters:

```
25
```

then after:

```
li $v0, 5
syscall
```

we have:

```
$v0 = 25
```

Important

If you want to keep the entered value, move it to another register.

For example:

```
li $v0, 5
syscall

move $t0, $v0
```

Now:

```
$t0
```

contains the user's input.

This is important because `$v0` will later be needed for other system calls.

8.4 System Call 10 – Exit

To terminate the program:

```
li $v0, 10
syscall
```

9. Part 8 – System Call Cheat Sheet

<code>\$v0</code>	Operation	Important Register
1	Print integer	<code>\$a0</code>
4	Print string	<code>\$a0</code>
5	Read integer	Result returned in <code>\$v0</code>
10	Exit program	None

Print a String

```
li $v0, 4
la $a0, message
syscall
```

Read an Integer

```
li $v0, 5
syscall
move $t0, $v0
```

Print an Integer

```
li $v0, 1
move $a0, $t0
syscall
```


Exit

```
li $v0, 10
syscall
```

10. Part 9 – Guided Activity: Input and Output**Task****Task**

Write a program that asks the user for a number and then displays it.
The interaction should look like:

```
Enter a number: 25
You entered: 25
```

Start with:

```
.data

prompt: .asciiz "Enter a number: "
msg:    .asciiz "You entered: "
```

Step 1 – Print the Prompt

```
li $v0, 4
la $a0, prompt
syscall
```

Step 2 – Read an Integer

```
li $v0, 5
syscall
```

Step 3 – Save the Input

```
move $t0, $v0
```

Step 4 – Print the Message

```
li $v0, 4
la $a0, msg
syscall
```

Step 5 – Print the Number

```
li $v0, 1
move $a0, $t0
syscall
```

Step 6 – Exit

```
li $v0, 10
syscall
```

11. Part 10 – Predict Before You Run

Consider:

```
li $v0, 5      to take input and read
syscall
move $t0, $v0

li $v0, 1      to print
move $a0, $t0
syscall
```

Suppose the user enters:

37

Predict Before You Run!

Predict the values:

Step	Register	Value
After input	\$v0	37
After move \$t0,\$v0	\$t0	37
Before print	\$a0	37

Now run the program.

Did your predictions match?

yes

12. Part 11 – Add and Sub with User Input

Now combine today's new concepts with what you learned in Lab 1.

Suppose the user enters:

10
5

We can save them in:

```
$t0 = 10
$t1 = 5
```

Then:

```
add $t2, $t0, $t1
```

gives:

```
$t2 = 15
```

And:

```
sub $t3, $t0, $t1
```

gives:

```
$t3 = 5
```

13. Part 12 – Guided Activity: Sum and Difference

Task

Write a program that asks for two integers.
The output should look like:

```
Enter first number: 20
Enter second number: 7

Sum = 27
Difference = 13
```

Required Steps

Your program should:

- | | |
|-----------------------------|---------|
| 1. Print the first prompt. | 1. done |
| 2. Read the first integer. | 2. done |
| 3. Save it in \$t0. | 3. done |
| 4. Print the second prompt. | 4. done |
| 5. Read the second integer. | 5. done |
| 6. Save it in \$t1. | 6. done |
| 7. Use <code>add</code> . | |
| 8. Use <code>sub</code> . | |
| 9. Print both results. | 7. done |
| 10. Exit. | |

Helpful Register Plan

Register	Purpose
\$t0	First input
\$t1	Second input
\$t2	Sum
\$t3	Difference

Task**You Try**

Complete the missing instructions:

```
# Read first number
li $v0, 5
syscall
-----

# Read second number
li $v0, 5
syscall
-----

# Calculate sum
-----

# Calculate difference
-----
```

14. Part 13 – Long Equations

Now we combine everything.

The user will enter values, and your program will use them in a longer equation.

Suppose the user enters:

```
T1 = 10
T2 = 5
T3 = 20
T4 = 4
```

We want to calculate:

$$T_1 + T_3 - (T_2 + T_2 + T_2 + T_2) + (T_4 + T_4 + T_4) - 15$$

Notice that we are **not using division or multiplication**.

Everything can be constructed using the instructions you already know:

```
add
sub
```

14.1 Step-by-Step Mathematics

First:

$$T_1 + T_3$$

$$10 + 20 = 30$$

Then:

$$T_2 + T_2 + T_2 + T_2$$

$$5 + 5 + 5 + 5 = 20$$

So:

$$30 - 20 = 10$$

Next:

$$T_4 + T_4 + T_4$$

$$4 + 4 + 4 = 12$$

Therefore:

$$10 + 12 = 22$$

Finally:

$$22 - 15 = 7$$

Important

Expected answer:

7

15. Part 14 – Final Programming Task

Task

User Input + Long Equation

Write a complete MIPS program that asks the user to enter four integers:

```
Enter T1:    10
Enter T2:     5
Enter T3:    20
Enter T4:     4
```

Your program must calculate:

$$T_1 + T_3 - (T_2 + T_2 + T_2 + T_2) + (T_4 + T_4 + T_4) - 15$$

and display:

```
The value of the expression is: _7_
```

15.1 Example Run

```
Enter T1: 10
Enter T2: 5
Enter T3: 20
Enter T4: 4

The value of the expression is: 7
```

15.2 Requirements

Your program must:

- use `.data`;
- declare prompts using `.asciiz`;
- use syscall 5 to read all four values;
- save the four inputs in registers;
- use `add` and `sub` to calculate the expression;
- use syscall 4 for the output message;
- use syscall 1 to display the final integer;
- use syscall 10 to exit;
- **not use `lw` or `sw`.**

16. Part 15 – Plan Before Coding

Important

Do not immediately start writing instructions.

First decide where each value will go.

Use the following register plan:

Register	Purpose
<code>\$t0</code>	T1
<code>\$t1</code>	T2
<code>\$t2</code>	T3
<code>\$t3</code>	T4
<code>\$t4</code>	Intermediate result
<code>\$t5</code>	Intermediate result

Now break the equation into smaller pieces.

Part A

Calculate:

$$T_1 + T_3$$

Part B

Calculate:

$$T_2 + T_2 + T_2 + T_2$$

Part C

Subtract Part B from Part A.

Part D

Calculate:

$$T_4 + T_4 + T_4$$

Part E

Add Part D.

Part F

Subtract 15.

17. Part 16 – Prediction Table

Before running your final program, use:

T1 = 10
T2 = 5
T3 = 20
T4 = 4

Predict the important registers.

Predict Before You Run!

Step	Operation	Register	Expected Value
1	T1 + T3	\$t4	_____ 30
2	T2 + T2	\$t5	_____ 10
3	+ T2	\$t5	_____ 15
4	+ T2	\$t5	_____ 20
5	+ T2	\$t5	_____ 25
6	Part A – Part B	\$t4	_____ 5
7	T4 + T4	\$t5	_____ 8
8	+ T4	\$t5	_____ 12
9	Add Part D	\$t4	_____ 17
10	Subtract 15	\$t4	_____ 2

Only after completing the table should you run the program.

18. Part 17 – Debugging Challenge

The following program is supposed to read two numbers and print their sum:

```

.data

prompt1: .asciiz "Enter first number: "
prompt2: .asciiz "Enter second number: "
msg:     .asciiz "Sum = "

.text

.globl main

main:

    li $v0, 4
    la $a0, prompt1
    syscall

    li $v0, 5          not saved anywhere
    syscall            add this
                       move $t0, $v0

    li $v0, 4
    la $a0, prompt2
    syscall

    li $v0, 5
    syscall            move $t1, $v0

    add $t2, $t0, $t1

    li $v0, 4
    la $a0, msg
    syscall

    li $v0, 1
    move $a0, $t2
    syscall

    li $v0, 10
    syscall

```

There is a problem.

Task

Find the Bug

Where are the two input values saved?

The program reads the first value into `$v0`, but then immediately uses `$v0` for another system call.

Fix the program so that:

```

first input -> $t0
second input -> $t1

```

Then calculate:

```

add $t2, $t0, $t1

```

12

19. Part 18 – Final Check

Before submitting, make sure your program can do the following.

Data

- Use `.data`
- Declare strings with `.ascii`
- Declare integers with `.word`
- Find your declared data in MARS memory

System Calls

- Print a string using syscall 4
- Read an integer using syscall 5
- Print an integer using syscall 1
- Exit using syscall 10

Arithmetic

- Use `add`
- Use `sub`
- Keep user inputs in registers
- Break a long equation into smaller calculations

Debugging

- Predict before running
- Step through instructions
- Check register values
- Identify where an incorrect value first appears

20. Key Idea From Today's Lab

You have now seen three important parts of a MIPS program.

1. Declared Data

`.data`

↓

Declared variables and strings

↓

Memory

2. User Input

USER



`syscall 5`



`$v0`



`$t-register`

3. Calculation and Output

Registers



`add / sub`



Result



`syscall 1`



Screen

Important

Declare → Input → Registers → Calculate → Display

Remember: `lw` and `sw` are intentionally not used in this lab. They will be introduced later when you are ready to explicitly move data between memory and registers.